



Kotlin
2nd Seminar

2026.05.03

코틀린 스터디

week2

Index

week2

- 변수
- 참조
- 번외) 스택 vs 힙
- 함수
- 문 vs 식(feat Java, c++)
- 클래스와 프로퍼티
- 커스텀 접근자
- 이넘 클래스와 이넘 상수 정의
- when
- 스마트 캐스트
- while && for 루프
- 번외 안내사항 3가지

코틀린 변수란? (val, var)

- val : 읽기 전용 참조
- var : 재대입 가능한 참조

```
data class User(var name: String)
```

```
fun main() {
```

```
    val user = User("윤지")
```

```
    user.name = "❤️서원❤️"
```

```
    // user = User("지민")
```

```
    var userB = User("지민")
```

```
    userB = User("현우")
```

```
    val queue = Queue()
```

```
    val a = queue
```

```
    a.enqueue(1)
```

```
    println(queue.size())
```

```
    val list = mutableListOf("one", "two", "three", "four")
```

```
    list.add("five")
```

```
    //list = mutableListOf("one", "two", "three", "four")
```

```
}
```


참조란? (feat 클로드)

```
data class User(var name: String)
```

```
fun main() {
```

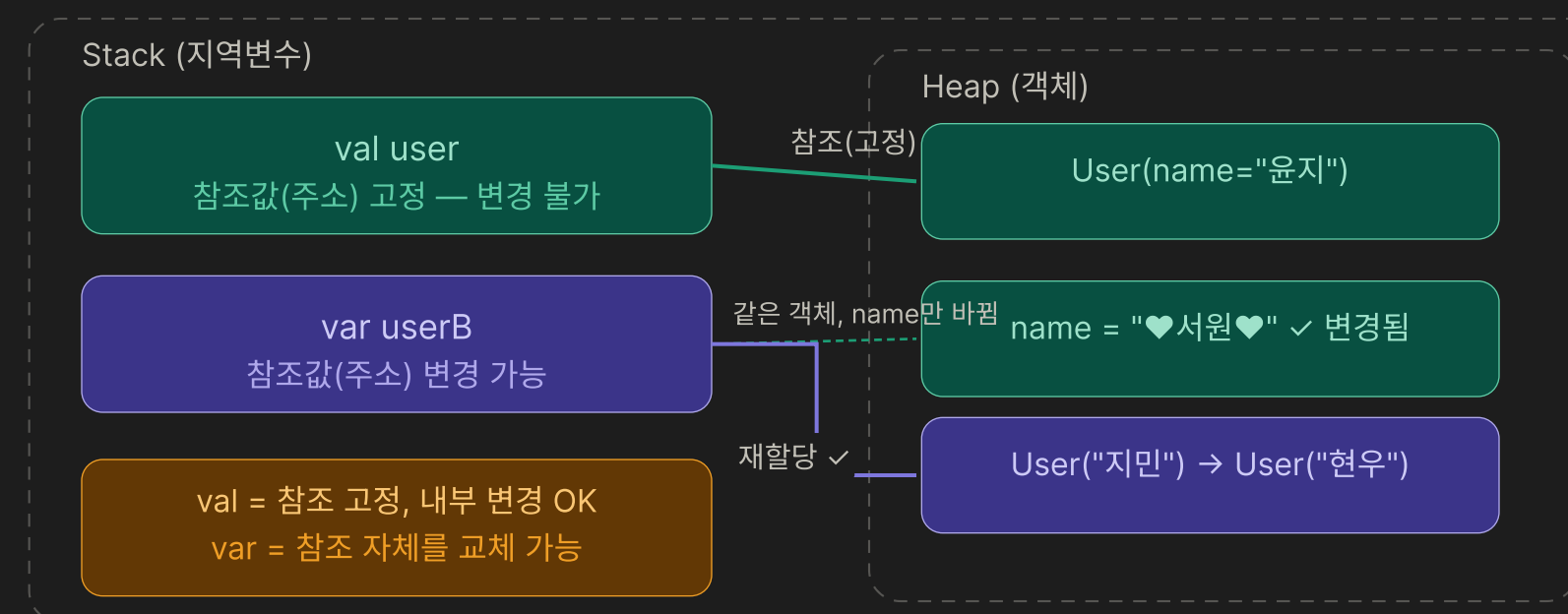
```
    val user = User("윤지")
    user.name = "♥서원♥"
    // user = User("지민")
```

```
    var userB = User("지민")
    userB = User("현우")
```

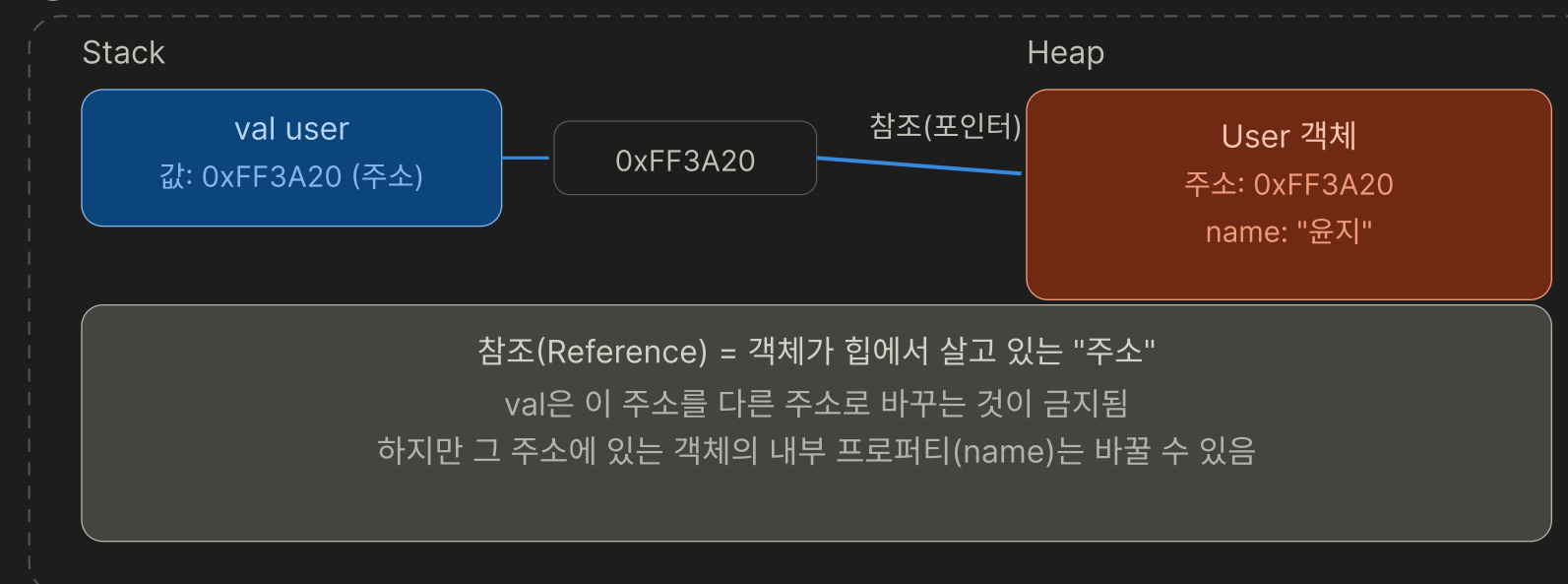
```
    val queue = Queue()
    val a = queue
    a.enqueue(1)
    println(queue.size())
```

```
    val list = mutableListOf("one", "two", "three", "four")
    list.add("five")
    //list = mutableListOf("one", "two", "three", "four")
```

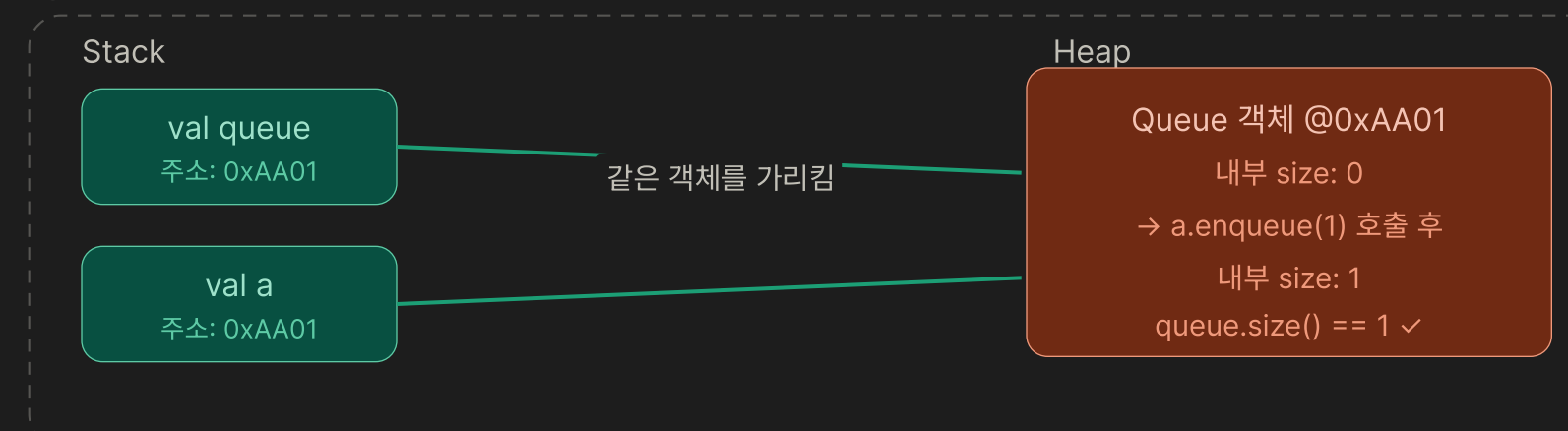
① val / var 차이



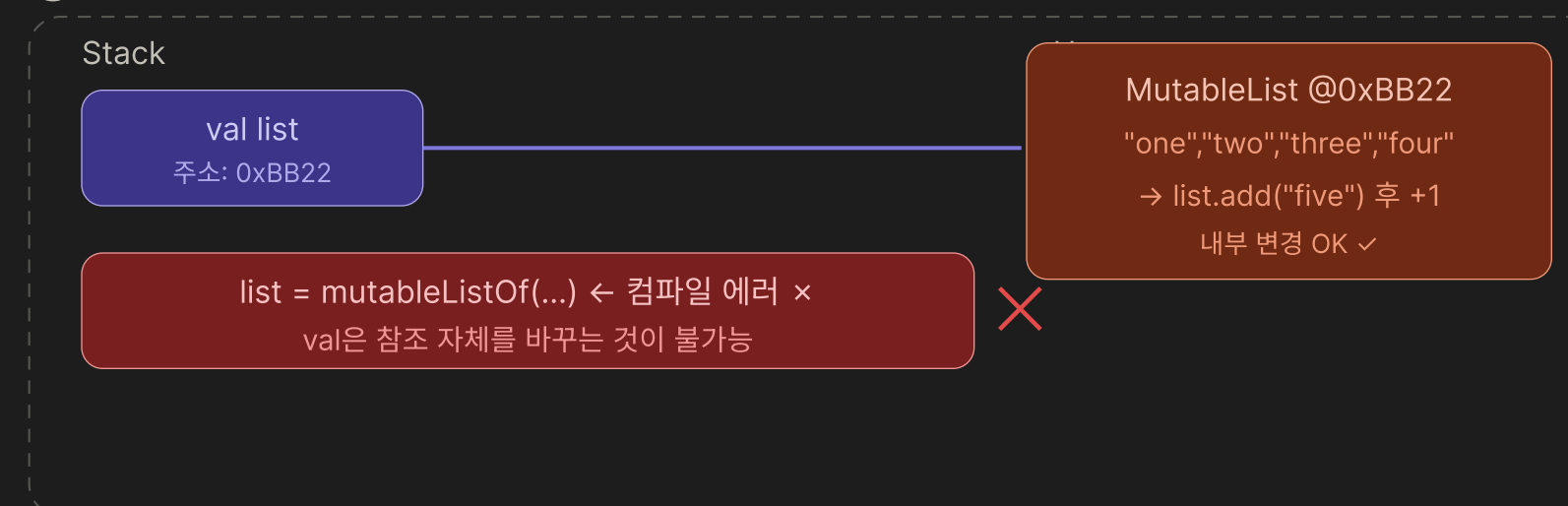
② 참조(Reference)란?



③ val a = queue / a.enqueue(1)



④ val list — 내부 변경 vs 재할당



번외) 스택 vs 힙 (feat 지민씨)

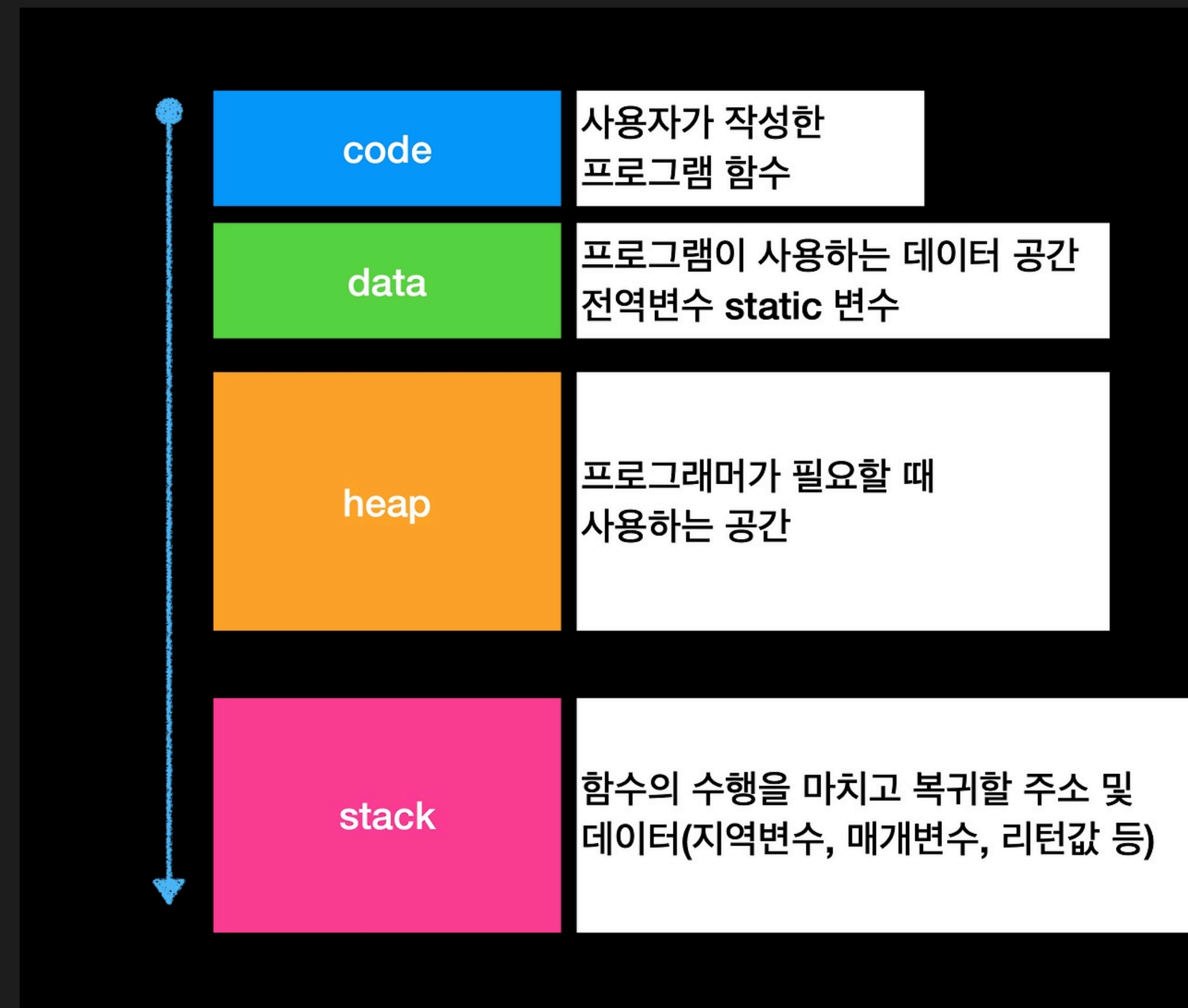
스택 : 실행 중인 함수의 정보가 저장됨

힙 : 함수에서 생성한 객체 정보가 저장됨

ex) User("지민")처럼 new로 만들어지는 모든 객체

=> 객체가 저장된 포인터를 변수가 갖고 있는 것임

전역 변수, 정적 변수는 데이터 영역에 저장됨.(OS 관점)



번외) JVM GC - JVM 메모리 구조



JVM 프로세스

- Heap
 - Young Generation
 - Eden
 - Survivor 0 / Survivor 1
 - Old Generation

- Java Stack
 - Stack Frame
 - Local Variable Array
 - Operand Stack
 - Dynamic Linking 정보
 - Return Address

- PC Register

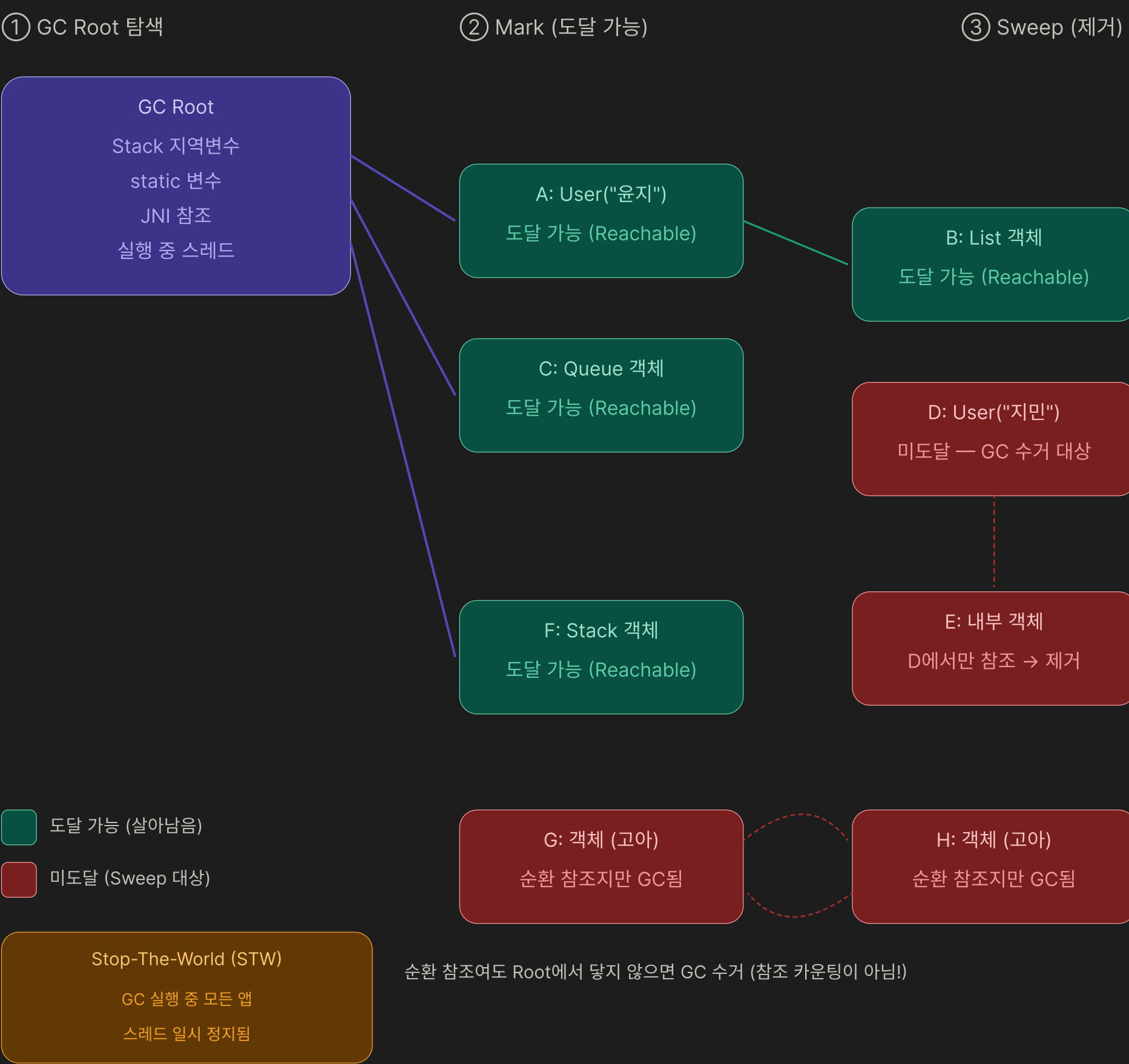
- Method Area
 - Java 8 이후 보통 Metaspace

- Runtime Constant Pool

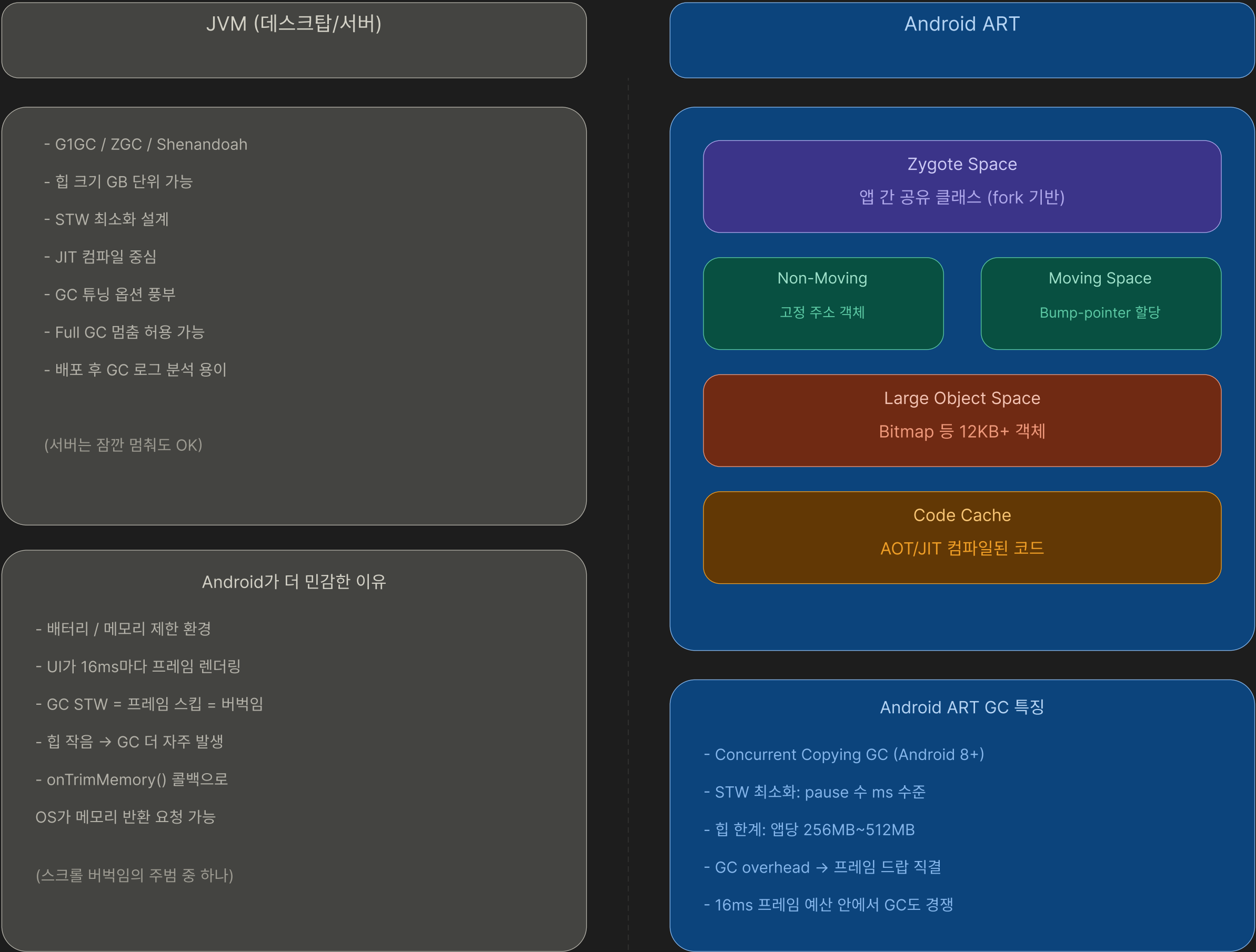
- Code Cache

- Native Memory
 - Metaspace
 - Direct Buffer
 - JNI 메모리
 - Native Library
 - Thread Stack 등

번외) JVM GC - JVM GC 동작 원리



번외) JVM GC - Android에서의 GC (ART 런타임)



번외) JVM GC - 개발자가 GC를 줄이는 방법

객체 생성 최소화

onDraw() / 루프 안 객체 생성 금지

BAD: `val paint = Paint()` // 매 프레임
GOOD: 클래스 필드로 한 번만 생성

문자열 연산

BAD: `"Hello " + name + "!"` (임시 객체)
GOOD: `"Hello $name!"` (Kotlin 최적화)

람다 / 고차함수

`inline fun`으로 람다 객체 생성 방지
Kotlin `inline` → 바이트코드 인라인

객체 재사용 (Object Pooling)

RecyclerView

ViewHolder 재사용으로 View 객체
반복 생성/GC 방지 — 핵심 패턴

Bitmap 재사용

`BitmapFactory.Options.inBitmap`
같은 크기 Bitmap 재사용 할당

Message 재사용

`Message.obtain()` — Handler 풀 사용
`new Message()` 직접 생성 피하기

메모리 누수 방지 (GC 못하게 막는 것)

Context 누수

static 변수에 Activity 참조 금지
Application Context 사용 권장

ViewModel / LiveData 누수

`observe()` → `lifecycleOwner` 전달
`viewLifecycleOwner` (Fragment 주의)

Coroutine 스코프

`viewModelScope` / `lifecycleScope` 사용
`GlobalScope` → 액티비티 파괴 후도 살아있음
LeakCanary로 누수 탐지

Compose / Kotlin 최적화

불필요한 리컴포지션 방지

`remember { }` 로 객체 캐싱
`@Stable` / `@Immutable` 어노테이션
→ 스마트 리컴포지션 유도

Sequence vs List

`list.filter().map()` → 중간 List 생성
`sequence.filter().map()` → 지연 평가
대용량 데이터일 때 GC 압력 감소

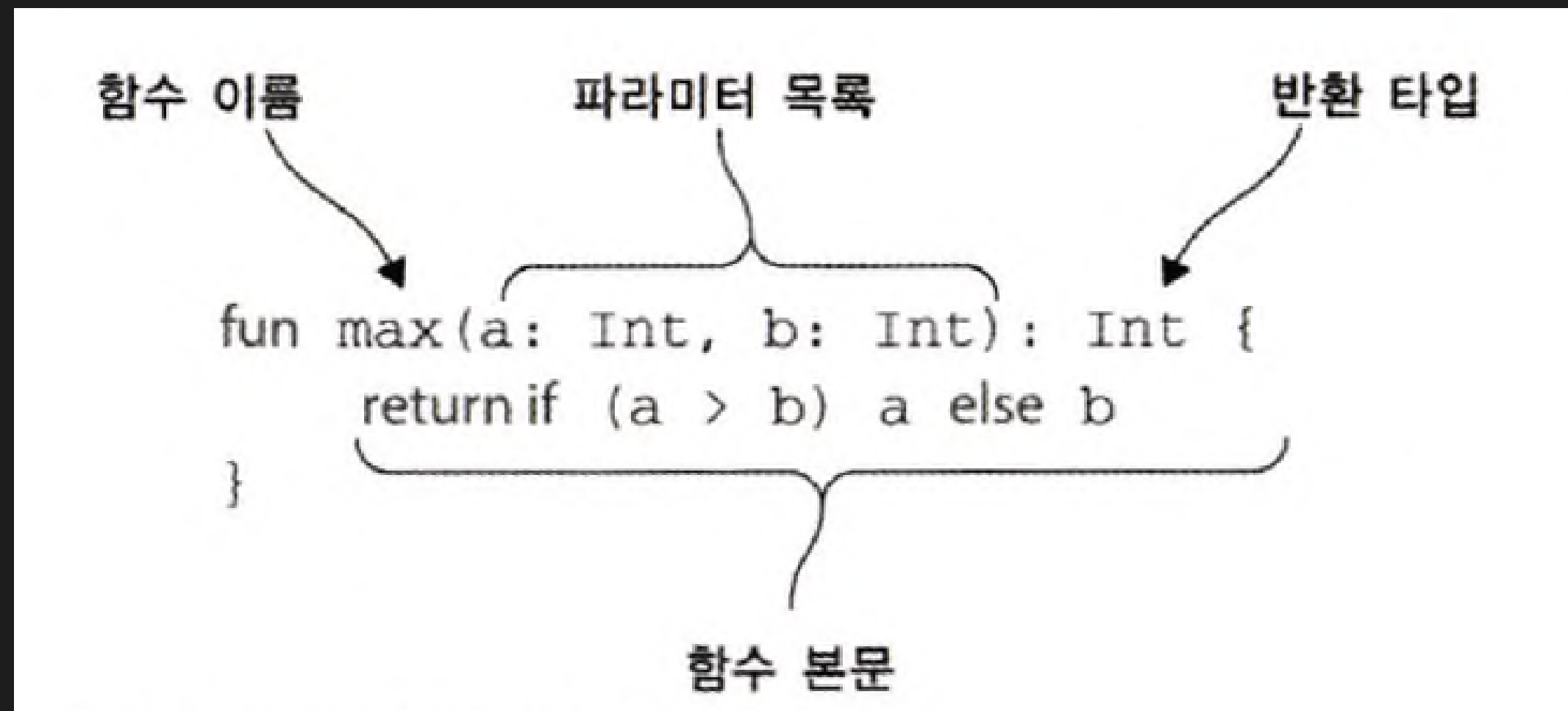
SparseArray / ArrayMap

HashMap 대신 → 박싱 객체 없음
정수 키일 때 GC 부담 크게 줄어듦

측정 먼저, 최적화 나중에 — 도구들

Android Studio Profiler → Memory 탭: GC 이벤트, 힙 할당 실시간 확인
LeakCanary → 메모리 누수 자동 탐지 및 스택 트레이스 출력
Perfetto / systrace → GC pause와 프레임 드랍 상관관계 분석
Compose Recomposition Counter → 불필요한 리컴포지션 시각화

코틀린 함수란?



color 코트 일부 발췌

```
fun rgb() = (r * 256 + g) * 256 + b  
fun printColor() = println("$this ${rgb()}")
```

위의 코드를 한 번 블록 함수로 변환해보면 컴파일러
자동 추론을 확인해볼 수 있습니다:)

식 vs 문 - Java

교재 79 쪽 블록을 꼭 읽어봐주세요:)

```
package chapter2;

public class expression {
    private static String name = "gmin";

    public static void main(String[] args) {
        // 자바는 코틀린과 달리 if가 문이어서, 값이 나오지 않음.
        if (name instanceof String) {           자바는 대입이 식이지만
            System.out.println(name);           if 조건은 Boolean만 허용!
        }
        System.out.println(name = "❤서원❤"); // 자바는 코틀린과 달리, 대입이 식임.
    }
}
```

식 vs 문 - C++

교재 79 쪽 블록을 꼭 읽어봐주세요:)

```
char* name = "현우";

if (name = "지민") {    // 대입식 "지민"이 non-null → 항상 true!
    printf("같다\n");    // 항상 실행됨
}
// name도 "지민"으로 바뀌어버림 - 의도하지 않은 부작용
```


식 vs 문 - 코틀린

교재 79 쪽 블록을 꼭 읽어봐주세요:)

```
package kos.chapter2
```

```
val name : String = "gmin"
```

코틀린은 대입이 문!!

```
fun main() {
```

```
    val result = if (name is String) name else "이름 없음"
```

```
    // if문이 식임을 보기 위해 억지로 작성된 코드임.
```

```
    // 현실적으로는 val result = name ?: "이름 없음"
```

```
    println(result)
```

```
    // val rename = result()
```

```
}
```

클래스와 프로퍼티 - Java

- 자바는 name 필드에 접근하기 위해 게터 함수 필요.
- 생성자 직접 작성 필수

```
package chapter2.member;

import kos.chapter2.member.Status;

public class Member {
    private String name;
    private final Status status;

    public Member(String name, Status status ){
        this.name = name;
        this.status = status;
    }

    public String getMember(){
        return name;
    }

    public void setMember(String name){
        this.name = name;
    }

    public Status getStatus() {
        return status;
    }
}
```

클래스와 프로퍼티 - 코틀린

- 프로퍼티 : 클래스와 데이터를 연관시키고, 접근 가능하게 만듦
- val : getter 자동 생성
- var : getter, setter 자동 생성
- 생성자 자동 생성

```
package kos.chapter2.member

sealed class Status{
    object Leader : Status()
    object Member : Status()
}

private class Member(
    val name : String,
    var status: Status // val, var 차이를 확인해주세요:)
) { }

fun main() {
    val member = Member("gmin", Status.Leader)
    println(member.name) // 코틀린에서 이걸 보는 순간 이름 게터를 호출합니다.
    println(member.status)

    member.status = Status.Member //코틀린이 자동으로 세터 호출함.
    println(member.status)

    //member.name = "smin" // 이걸 왜 오류일까요?
}
```


클래스와 프로퍼티 - 커스텀 접근자

- 필드 저장 없이 게터 호출할 때마다 age 보고 계산함.

```
package kos.chapter2.member

sealed class Status{
    object Leader : Status()
    object Member : Status()
}

private class Member(
    val name : String,
    var status: Status // val, var 차이를 확인해주세요:)
) { }

fun main() {
    val member = Member("gmin", Status.Leader)
    println(member.name) // 코틀린에서 이걸 보는 순간 이름 게터를 호출합니다.
    println(member.status)

    member.status = Status.Member //코틀린이 자동으로 세터 호출함.
    println(member.status)

    //member.name = "smin" // 이걸 왜 오류일까요?
}
```

클래스와 프로퍼티 - 이넘 클래스

- Red...는 Color 클래스의 미리 정의된 인스턴스임.
- r,g,b 프로퍼티도 가짐.
- 함수 호출 가능한 객체임
- 인스턴스가 공유하는 멤버 함수가 있음.

```
package kos.chapter2.color

enum class Color(
    val r : Int,
    val g : Int,
    val b : Int
){
    RED(255, 0, 0),
    GREEN(0, 255, 0),
    BLUE(0, 0, 255);

    fun rgb() = (r * 256 + g) * 256 + b
    fun printColor() = println("$this ${rgb()}")
}

fun main(){
    println(Color.RED.rgb())
    Color.BLUE.printColor()
}
```

이넘이 왜 필요한가?

- 타입 안정성을 위해서임.
- 뒤에 나올 when과 사용하기 용의함.

```
// 상수로만 쓰면
const val RED = 0
const val GREEN = 1
const val BLUE = 2

fun printColor(color: Int) { // Int면 뭐든 들어올 수 있음
    println(color)
}

printColor(999) // 컴파일 OK - 근데 뭔지 모름
printColor(-1) // 컴파일 OK - 말이 안 되는 값
```

```
fun printColor(color: Color) { // Color 타입만 허용
    println(color)
}

printColor(999) // 컴파일 에러!
printColor(Color.RED) // 이것만 가능

when (color) {
    Color.RED -> "빨강"
    Color.GREEN -> "초록"
    Color.BLUE -> "파랑"
    // 빠뜨리면 컴파일러가 경고
}
```


when

- 자바에서는 switch라고 생각하면 됨.

```
package kos.chapter2.color
```

```
fun getMnemonic(color: Color): String {  
    return when (color) {  
        Color.RED -> "red"  
        Color.BLUE -> "blue"  
        Color.GREEN -> "green"  
    }  
}
```

```
fun main() {  
    println(getMnemonic(Color.BLUE))  
}
```

식(반환값)이 있어야 return /
val = when 모든 케이스 강제

when(인자 있음, 인자 없음)

```
package kos.chapter2

import kos.chapter2.color.Color

fun mix(c1: Color, c2: Color) =
    when (setOf(c1, c2)) {
        setOf(Color.RED, Color.BLUE) ->
            Color.PURPLE
        else -> throw Exception("Dirty color") //
set 항상 강제
    }

fun main() {
    println(mix(Color.BLUE, Color.RED))
}
```

```
package kos.chapter2

import kos.chapter2.color.Color

fun mixOptimized (c1: Color, c2: Color) =
    when { // 인자가 없는 경우 내부 조건은 boolean 형식임. else
강제
        (c1 == Color.RED && c2 == Color.BLUE ) ->
            "purple"
        else -> throw Exception("Dirty color")
    }

fun main() {
    println(mixOptimized(Color.BLUE, Color.RED))
}
```


스마트 캐스트

- 타입 검사와 타입 캐스트 조합
- is :타입 체크(스마트 캐스트) -컴파일러
- as : 명시적 캐스트 - 런타임

```
package kos.chapter2

fun eval(e: Expr): Int {
    if (e is Num) {
        val n = e as Num // 불 필요함.
        return n.value
    }
    if (e is Sum) {
        return eval(e.right) + eval(e.left)
    }
    throw IllegalArgumentException("Unknown expression")
}

fun main() {
    println(eval(Sum(Sum(Num(1), Num(2)), Num(4))))
}
```

스마트 캐스트 - 실제 사례

```
val googleLoginState by viewModel.googleLoginState
    .collectAsStateWithLifecycle()
```

```
is SocialLoginState.Success -> {  
    val result =  
        (googleLoginState as SocialLoginState.Success).result
```

매번 getter 호출



```
when (val state = googleLoginState) {  
    is SocialLoginState.Success -> {  
        val result = state.result // as 불필요!  
    }  
    is SocialLoginState.Error -> {  
        Log.e("GoogleLogin", state.message) // as 불필요!  
    }  
}
```

while

```
fun main() {  
    var i = 1  
    while (i <= 10) {  
        println(i)  
        i++  
    }  
    // 1 2 3 4 5 6 7 8 9 10  
  
    var j = 10  
    do {  
        println(j)  
        j--  
    } while (j > 0)  
    // 10 9 8 7 6 5 4 3 2 1  
}
```


for - 퍼즈버즈

```
fun fizzBuzz(i: Int) = when {  
    i % 15 == 0 -> "피즈버즈 "  
    i % 3 == 0  -> "피즈 "  
    i % 5 == 0  -> "버즈 "  
    else        -> "$i "  
}  
  
fun main() {  
    // 기본 범위 순방향  
    for (i in 1..100) {  
        print(fizzBuzz(i))  
    }  
  
    println()  
  
    for (i in 100 downTo 1 step 2) {  
        print(fizzBuzz(i))  
    }  
  
    println()  
  
    for (i in 1 until 10) {  
        print("$i ")  
    }  
}
```

논의사항 1

한 번 날 좋을 때 한강 스테디를
개최할 예정인데
언제가 좋아요...?

우천시 취소,
러브버그 많아도 취소



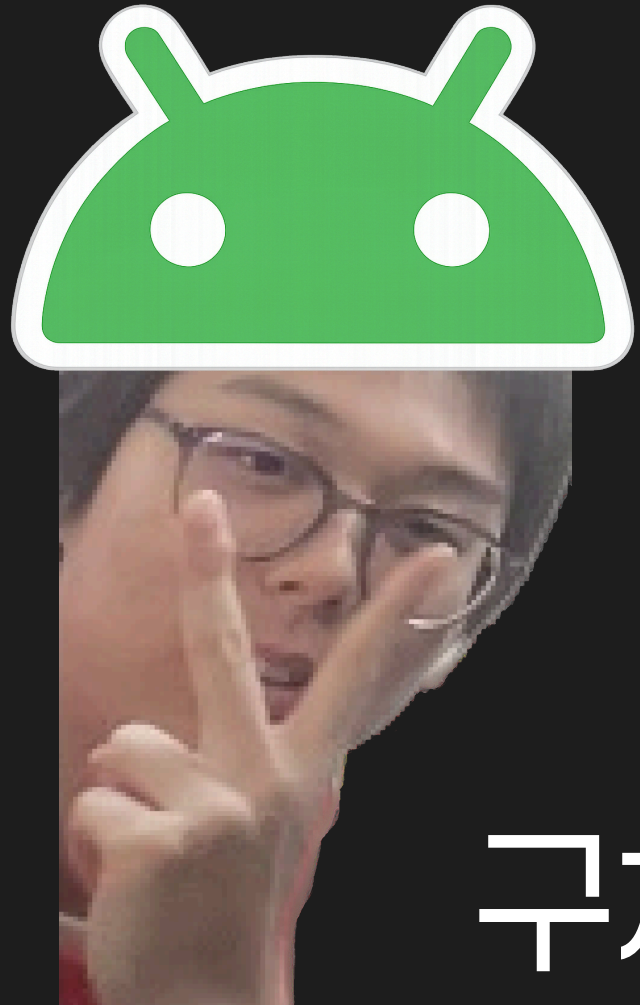
논의사항 2

프론트 ui 개발 일정 짜기

pm 요청사항

오늘 스터디 끝나고 회의 합니당....

논의사항 3



프론트 ui 개발 일정 회의 후
지민씨와 현우씨와의 1:1 회의~!

구체적으로 지민씨가 다음주 현우씨 할 내용들 알려줄거예요!

Q and A

모르면 클로드가 답해드립니다.
지민씨가 더 디테일하게 알려줄거예요
제 답변을 확신하지 마세요